Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики

ЗВІТ

з лабораторної роботи №2
з дисципліни «Розробка мультимедійних систем»

на тему: «Розробка мультимедійного застосунку
з використанням сучасних технологій»
Варіант 1. Медіаплеєр

Виконав:
студент групи ІПС-41
Іванченко Владислав Віталійович

Перевірив(ла):
_____

Київ 2026

Зміст

1. Мета і завдання
2. Засоби розробки
3. Виконання вимог варіанта
4. Додаткові можливості та інтерфейс
5. Структура програми
6. Шлях даних від введення до результату
7. Робота з різними типами вхідних даних
8. Тестування
9. Висновки

1. Мета і завдання

Мета роботи: розробити desktop-застосунок для відтворення аудіо та відео і перевірити його на коректних і некоректних вхідних даних.

Вимоги варіанта 1: відкриття локального медіафайлу; Play / Pause та Stop; перехід вперед і назад; переміщення по запису за допомогою progress bar; відображення поточного та загального часу; регулювання гучності; підтримка щонайменше двох аудіоформатів і двох відеоформатів; відтворення медіаконтенту за URL.

2. Засоби розробки

Засіб	Для чого
Python 3.13	мова реалізації
PySide6 6.12 (Qt 6.12)	вікно і віджети (Qt Widgets), відтворення (Qt Multimedia), рендер SVG-іконок (Qt SVG)
FFmpeg (бекенд Qt Multimedia)	читання контейнерів і декодування аудіо та відео; входить у PySide6, окремо не встановлюється
pytest	автоматичні тести
FFmpeg (утиліта)	генерація тестових медіафайлів

Плеєр не має власного декодера: Qt Multimedia вже вміє розбирати й декодувати всі потрібні формати, тому задача застосунку в тому, щоб правильно передати йому джерело і показати користувачеві стан відтворення. Уся програма займає один файл `player.py` (430 рядків разом зі стилями й іконками) і має одну залежність.

3. Виконання вимог варіанта

Вимога	Реалізація
Відкриття локального файлу	кнопка «Відкрити файли» або `Ctrl+O` (можна вибрати кілька), перетягування файлів у вікно, шлях у командному рядку
Play / Pause та Stop	кругла кнопка відтворення працює як перемикач, її іконка й підказка змінюються разом зі станом плеєра; кнопка Stop повертає позицію на початок; пробіл
Перехід вперед і назад	кнопки та клавіші ← і → перемотують на 5 с; кнопки і клавіші N / P перемикають файли в списку
Переміщення по запису	смуга прогресу: перетягування повзунка, клік у будь-яку точку смуги, клавіші; при наведенні курсора над смугою видно таймкод цієї точки
Поточний і загальний час	дві мітки у форматі `мм:сс`, для записів довших за годину `г:мм:сс`
Гучність	повзунок від 0 до 100 %, кнопка і клавіша M вимикають звук без втрати рівня гучності
Формати	перевірено тестами: аудіо WAV і MP3, відео MP4 (H.264 + AAC), WebM (VP8 + Vorbis), MKV (H.264 + MP3). Відкриваються й інші формати, які декодує FFmpeg (OGG, FLAC, M4A, AVI, MOV)
Відтворення за URL	кнопка «Відкрити посилання» або `Ctrl+U`: http, https, rtsp, rtmp, ftp, file

4. Додаткові можливості та інтерфейс

Можливість	Як працює
Таймкод під курсором	при наведенні на смугу прогресу над нею з'являється час відповідної точки; клік переносить відтворення туди ж, і повзунок одразу можна тягнути далі
Список відтворення	кілька файлів з діалогу або перетягуванням; подвійний клік вмикає файл; після кінця файлу автоматично грає наступний; панель відкривається сама, коли файлів більше одного, і кнопкою або клавішею L
Швидкість відтворення	кнопка перемикає 1×, 1.25×, 1.5×, 2×, 0.5×, 0.75×
Повтор	кнопка або клавіша R зациклює поточний файл
Повний екран	кнопка або клавіша F, вихід клавішею Esc
Збереження кадру	кнопка або `Ctrl+S` зберігає поточний кадр відео в PNG у папку «Зображення», ім'я містить назву файлу і час кадру

Зрозумілі помилки	замість технічного тексту Qt користувач бачить причину українською (файлу немає, це не медіафайл, посилання недоступне); технічна деталь лишається в підказці рядка стану
Стани екрана	порожній стан з підказкою, відео, обкладинка з назвою для аудіо, пояснення помилки
Клавіатура і доступність	усі дії мають клавіші; кожна кнопка має підказку з клавішею і назву для екранного диктора; фокус видно за кольоровою рамкою

Інтерфейс темний, щоб не відволікати від відео. Бурштиновий колір позначає лише головне: кнопку відтворення, пройдену частину запису і увімкнені перемикачі (список, повтор). Решта кнопок без рамок, другорядний текст приглушений, час набрано моноширинним шрифтом, щоб цифри не стрибали під час відтворення. Іконки векторні, рендеряться з SVG у потрібному кольорі. Увесь вигляд задає один Qt Style Sheet, логіка від нього не залежить.

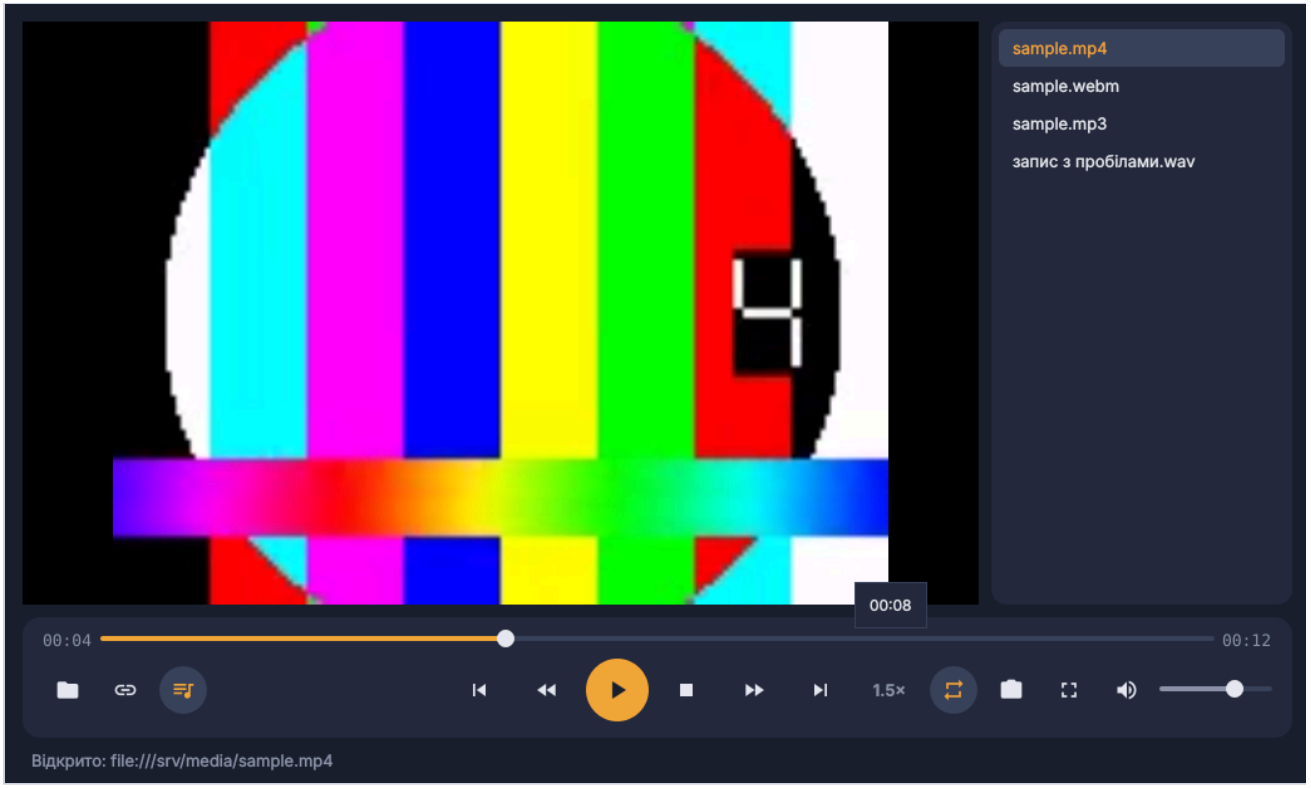


Рис. 1. Відтворення відео: відкрито список із чотирьох файлів, курсор над смугою показує таймкод 00:08, увімкнені швидкість 1.5× і повтор

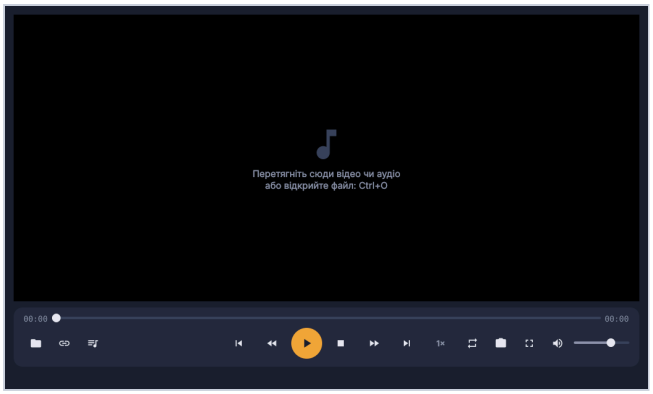


Рис. 2. Порожній стан

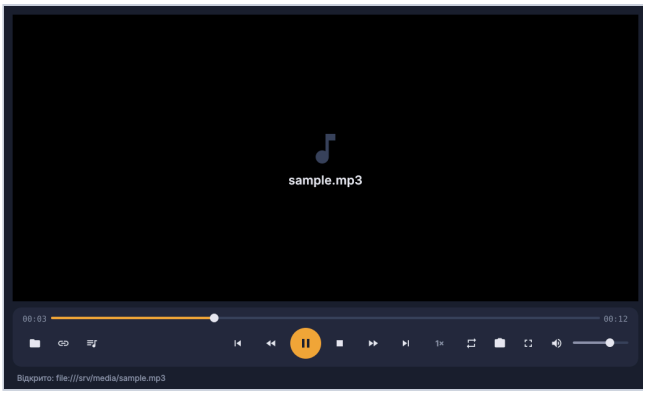


Рис. 3. Аудіофайл: обкладинка з назвою

5. Структура програми

```
ЛР2_медіаплеєр/
  player.py            застосунок
  tests/
    conftest.py        спільні фікстури: QApplication, вікно плеєра
    test_player.py     66 тестів
    media/             тестові файли: WAV, MP3, MP4, WebM, MKV, файл з кирилицею і
пробілами в назві
      bad/             некоректні файли: порожній, випадкові байти, текст, обрізаний,
.txt, .png
  inject_faults.py     перевірка тестів ін'єкцією помилок
  requirements.txt
  README.md
```

`player.py` складається з чотирьох частин.

Частина	Що в ній	Призначення
Константи	`SKIP_MS`, `SPEEDS`, `ERRORS`, палітра, `ICONS`, `STYLE`	налаштування в одному місці: крок перемотування, швидкості, тексти помилок, кольори, контури іконок, таблиця стилів
Функції	`icon()`, `fmt_time()`, `to_url()`	чисті перетворення без стану: SVG у іконку, мілісекунди в рядок часу, введений рядок у `QUrl`
Клас `SeekSlider`	нащадок `QSlider`	смуга прогресу з таймкодом під курсором і переходом кліком
Клас `Player`	нащадок `QWidget`	вікно: будує інтерфейс, тримає список відтворення, з'єднує кнопки з плеєром і плеєр з інтерфейсом

Методи `Player` згруповані за роллю:

- джерела: `add_sources`, `open_source`, `play_index`, `open_file_dialog`, `open_url_dialog`, `dropEvent`;

- керування за командою користувача: `toggle_play`, `seek`, `cycle_speed`, `toggle_fullscreen`, `snapshot`;
- реакція на сигнали плеєра: `on_position`, `on_duration`, `on_state`, `on_media_status`, `on_error`, `on_muted`.

Обробники `on_*` нічого не вирішують самі: вони лише переносять стан `QMediaPlayer` в інтерфейс. Тому інтерфейс не може «розійтися» з плеєром: усе, що бачить користувач, береться з плеєра.

6. Шлях даних від введення до результату

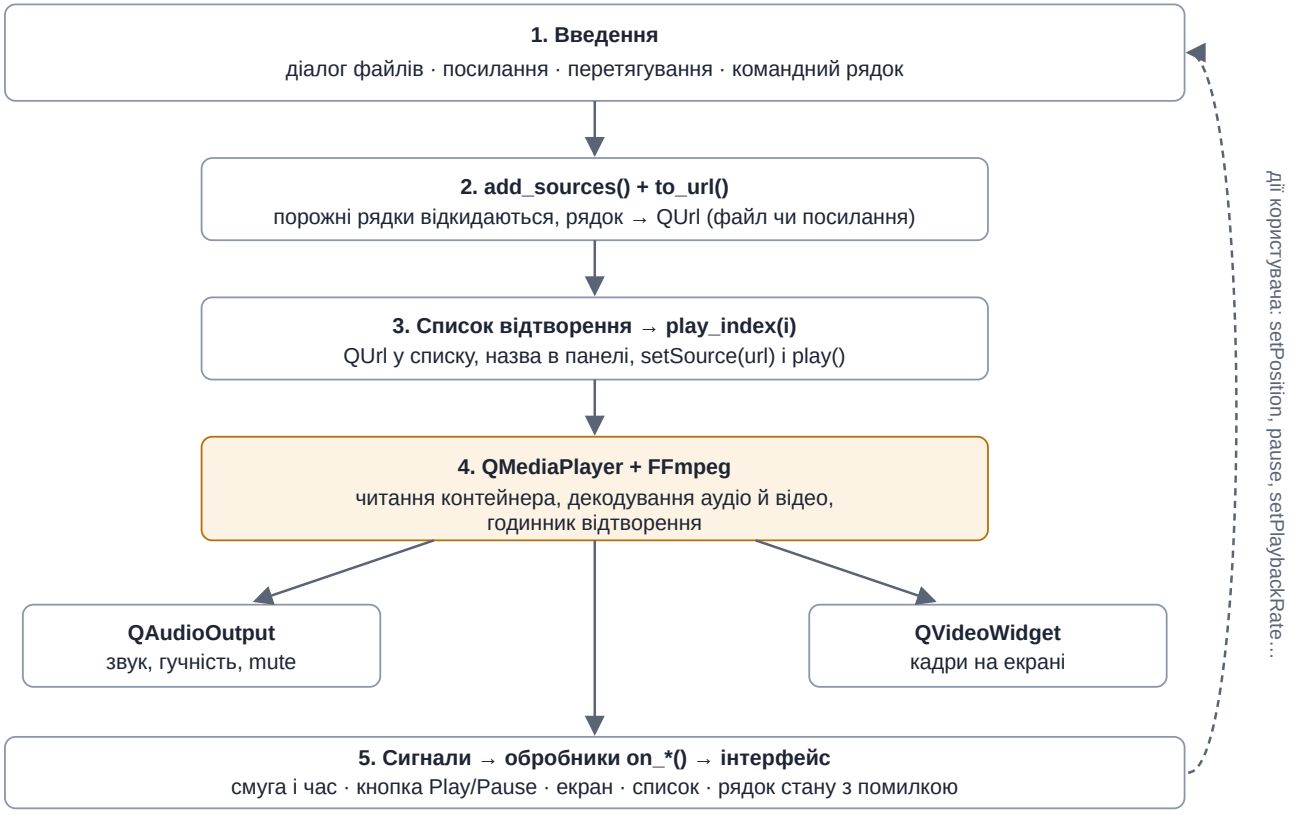


Рис. 4. Шлях даних у програмі

Кроки 1 і 2. Від введеного рядка до QUrl

Усі способи відкриття (діалог, посилання, перетягування, командний рядок) зводяться до одного методу `add_sources()`, який отримує список рядків. Порожні рядки відкидаються, решта перетворюється функцією `to_url()`. Правило просте: якщо в рядку є `://`, це посилання; інакше локальний шлях. Перевіряти схему через `QUrl(text).scheme()` не можна: шлях Windows `C:\video.mp4` Qt розбирає як посилання зі схемою `c`.

```
110  def to_url(text: str) -> QUrl:
111      """Рядок з поля вводу -> QUrl: усе з '://' — посилання (http, rtsp, file...),
решта — локальний шлях."""
112      text = text.strip()
113      # не QUrl(text).scheme(): шлях 'C:\video.mp4' Qt розбирає як посилання зі схемою
'c'
114      return QUrl(text) if "://" in text else QUrl.fromLocalFile(text)
```

```
305  def add_sources(self, texts):
306      """Додати файли/посилання в кінець списку і почати відтворення першого з
доданих."""
307      urls = [to_url(t) for t in texts if t.strip()]
308      if not urls:
309          return
310      first = len(self.playlist)
311      for url in urls:
312          self.playlist.append(url)
313          self.list.addItem(url.fileName() or url.toDisplayString())
314      if len(self.playlist) > 1:
315          self.list_btn.setChecked(True)
316      self.play_index(first)
```

Крок 3. Список відтворення і запуск

`play_index()` перевіряє межі списку (тому «попередній» на першому файлі і «наступний» на останньому нічого не роблять), підсвічує рядок у панелі, передає `QUrl` плеєру і запускає відтворення. Заголовок вікна, обкладинка і рядок стану оновлюються тут же.

```
322  def play_index(self, i: int):
323      if not 0 <= i < len(self.playlist):
324          return
325      self.index = i
326      url = self.playlist[i]
327      name = self.list.item(i).text()
328      self.list.setCurrentRow(i)
329      self.player.setSource(url)
330      self.setWindowTitle(f"{name} — Медіаплеєр")
331      self.cover.setText(name)
332      self.cover.show()
333      self.hint.hide()
334      self.status.setText(f"Відкрито: {url.toDisplayString()}")
335      self.player.play()
```

Крок 4. Декодування

Після `setSource()` Qt Multimedia у фоновому потоці відкриває файл чи посилання через FFmpeg, визначає контейнер і кодеки, декодує звук у `QAudioOutput` і кадри в `QVideoWidget`. Головний потік у цей час не блокується. Про все, що відбувається, плеєр повідомляє сигналами.

Крок 5. Сигнали плеєра і дії користувача

Зв'язки створюються один раз у конструкторі. Дії користувача викликають методи плеєра, а сигнали плеєра оновлюють інтерфейс:

```
270  self.pos_slider.actionTriggered.connect(
271      lambda _: self.player.setPosition(self.pos_slider.sliderPosition()))
272  self.vol_slider.valueChanged.connect(lambda v: self.audio.setVolume(v / 100))
273  self.audio.mutedChanged.connect(self.on_muted)
274  self.player.positionChanged.connect(self.on_position)
275  self.player.durationChanged.connect(self.on_duration)
276  self.player.playbackStateChanged.connect(self.on_state)
277  self.player.mediaStatusChanged.connect(self.on_media_status)
278  self.player.hasVideoChanged.connect(lambda has: self.stack.setCurrentWidget(
279      self.video if has else self.placeholder))
280  self.player.errorOccurred.connect(self.on_error)
```

Коли файл закінчується, `on_media_status` вмикає наступний у списку. Якщо увімкнено повтор, плеєр сам починає файл спочатку і до кінця не доходить.

```
399  def on_media_status(self, status):
400      if status == QMediaPlayer.EndOfMedia:  # з увімкненим повтором Qt сюди не
доходить
401          self.play_index(self.index + 1)  # саме синхронно: відкладений setSource тут
зависає в Qt FFmpeg
```

Смуга прогресу з таймкодом

Щоб показати час під курсором, координату x треба перевести в мілісекунди. Ділити x на ширину віджета неточно, бо повзунок має власну ширину і не заходить за краї жолоба. `value_at()` бере справжні межі жолоба й повзунка з поточного стилю і рахує позицію стандартною функцією Qt:

```
125  def value_at(self, x: int) -> int:
126      """Координата x у віджеті -> позиція в мс, з урахуванням ширини повзунка."""
127      opt = QStyleOptionSlider()
128      self.initStyleOption(opt)
129      groove = self.style().subControlRect(QStyle.CC_Slider, opt,
QStyle.SC_SliderGroove, self)
130      handle = self.style().subControlRect(QStyle.CC_Slider, opt,
QStyle.SC_SliderHandle, self)
131      span = groove.width() - handle.width()
132      return QStyle.sliderValueFromPosition(self.minimum(), self.maximum(),
133                                            x - groove.x() - handle.width() // 2, span)
```

Підказка з'являється над смугою. Qt сам зсуває підказку донизу на 16-21 піксель, і якщо вона лягає під курсор, Qt одразу її ховає, тому точка показу піднята з запасом. Поки файл не відкрито, тривалість дорівнює нулю, і підказка не показується.

```
135  def show_time(self, x: int):
136      if self.maximum() > 0:  # поки медіа не відкрите, показувати нічого
137          # Над смугою. Qt сам додає зсув +16…21 px донизу; якщо підказка ляже під
курсор,
138          # Qt одразу її сховає, тому піднімаємо з запасом
139          QToolTip.showText(self.mapToGlobal(QPoint(x - 22, -62)),
fmt_time(self.value_at(x)), self)
```

Клік ставить повзунок у точку кліку і викликає ту саму дію, що й перетягування. Плеєр перемотує, а повзунок опиняється під курсором, тож його можна одразу тягнути.

```
152  def mousePressEvent(self, event):
153      if event.button() == Qt.LeftButton and self.maximum() > 0:
154          self.setSliderPosition(self.value_at(int(event.position().x())))
155          self.triggerAction(QAbstractSlider.SliderMove)  # -> плеєр перемотує сюди
156      super().mousePressEvent(event)  # повзунок тепер під курсором, тож його можна
одразу тягнути
```

Обробка помилок

Qt повідомляє тип помилки і технічний текст англійською. Тип перетворюється на пояснення для користувача за таблицею `ERRORS`. Для посилань `ResourceError` означає недоступну адресу, а не відсутній файл, тому для них текст свій. Екран переходить у заставку з поясненням, а технічний текст лишається в підказці рядка стану.

```
 21  ERRORS = {
 22      QMediaPlayer.ResourceError: "файл не знайдено або він пошкоджений",
 23      QMediaPlayer.FormatError: "це не медіафайл або його формат не підтримується",
 24      QMediaPlayer.NetworkError: "посилання недоступне",
 25      QMediaPlayer.AccessDeniedError: "немає доступу до файлу",
 26  }
```

```
403  def on_error(self, error, message: str):
404      reason = ERRORS.get(error, message)
405      if error == QMediaPlayer.ResourceError and not
self.player.source().isLocalFile():
406          reason = "посилання недоступне або за ним немає файлу"
407      self.status.setText(f"Помилка: {reason}")
408      self.status.setToolTip(message)  # технічна деталь від Qt/FFmpeg
409      # на заставку екран перемикає hasVideoChanged: Qt сам скидає відео при
непридатному джерелі
410      self.hint.setText(f"Не вдалося відтворити: {reason}")
411      self.hint.show()
```

7. Робота з різними типами вхідних даних

Застосунок запускався з кожним вхідним значенням із таблиці. Для кожного зафіксовано стан плеєра (`mediaStatus`, `error`), тривалість і повідомлення, яке бачить користувач. Більшість випадків також перевіряють автоматичні тести (розділ 8).

Вхідні дані	Стан плеєра	Що бачить користувач
Коректні		
sample.wav, sample.mp3	BufferedMedia, без помилок, 12 с	звук, обкладинка з назвою файлу, час 00:12
sample.mp4, sample.webm, sample.mkv	BufferedMedia, є відео, 12 с	відео, час 00:12
«запис з пробілами.wav» (кирилиця і пробіли в назві)	BufferedMedia, 12 с	відтворюється, назва показується без спотворень
http://127.0.0.1:…/sample.mp4	BufferedMedia, є відео	відео з мережі
« …/sample.mp3 » (пробіли довкола шляху)	BufferedMedia	пробіли обрізаються, файл відкривається
Некоректні		
неіснуючий файл nope.mp4	InvalidMedia, ResourceError	«Помилка: файл не знайдено або він пошкоджений»
порожній файл empty.mp4 (0 байт)	InvalidMedia, FormatError	«Помилка: це не медіафайл або його формат не підтримується»
garbage.mp4: 50 КБ випадкових байтів	InvalidMedia, FormatError	
text.mp3: текст з розширенням .mp3	InvalidMedia, FormatError	
notes.txt	InvalidMedia, FormatError	
посилання на неіснуючий файл (HTTP 404)	InvalidMedia, ResourceError	«Помилка: посилання недоступне або за ним немає файлу»
посилання на закритий порт	InvalidMedia, ResourceError	
ftp://example.com/a.mp4	InvalidMedia, ResourceError	

порожній рядок або лише пробіли	NoMedia	нічого не відбувається, попередній стан не змінюється
Граничні випадки		
truncated.mp4: перші 37 % справжнього MP4	BufferedMedia, 12 с, без помилки	відтворюється частина, що збереглася; тривалість береться з заголовка файлу, тому лічильник доходить до 00:12
image.png	EndOfMedia, є відео, 0 с	FFmpeg відкриває картинку як відео з одного кадру: кадр показується, відтворення одразу завершується

За жодних вхідних даних програма не падає і не зависає. Після помилки можна одразу відкрити інший файл: заставка з поясненням ховається, і відтворення йде як звичайно (це окремо перевіряє тест `test_recovers_after_error`).

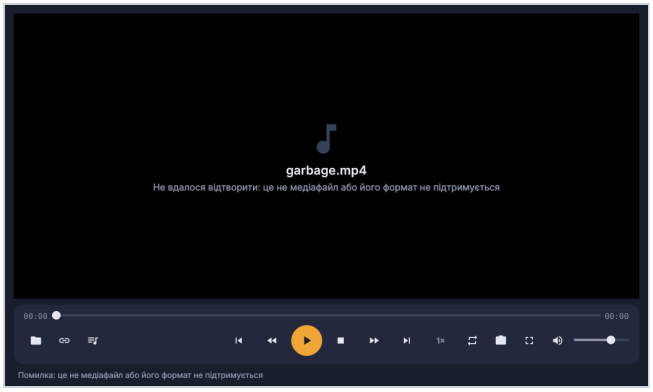


Рис. 5. Випадкові байти з розширенням .mp4

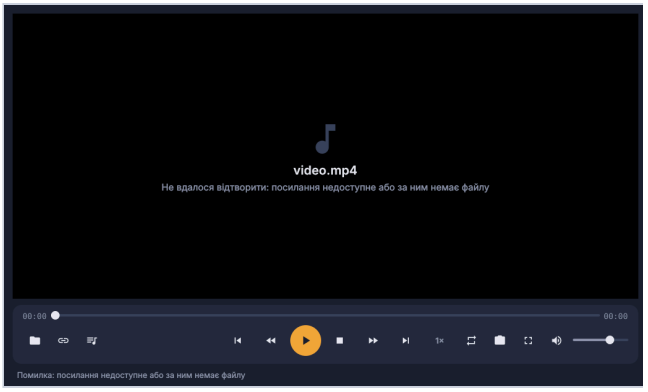


Рис. 6. Недоступне посилання

8. Тестування

66 тестів pytest працюють зі справжніми медіафайлами і справжнім `QMediaPlayer`, без підміни бекенду. Вікно створюється заново для кожного тесту і після тесту звільняє декодер. Для перевірки відтворення за URL тести піднімають локальний HTTP-сервер в окремому процесі.

Група	Тестів	Що перевіряється
`fmt_time`, `to_url`	20	межі хвилини й години, від'ємний час; http, https, rtsp, ftp, великі літери в схемі, шляхи Linux і Windows, кирилиця, пробіли, `file://`
Формати й екран	9	шість файлів грають з правильною тривалістю; порожній стан, відео на екрані, обкладинка для аудіо, заголовок вікна; відтворення за URL

Керування	9	Play/Pause (на паузі позиція стоїть), Stop, ±5 с і межі запису, перемотування смугою, смуга йде за відтворенням і не смикається під час перетягування, гучність, вимкнення звуку
Таймкод і клік по смузі	4	час під курсором відповідає місцю на смузі; без файлу підказки немає; крайні точки смуги дають 0 і повну тривалість; клік переносить у точку кліку
Некоректні дані	10	п'ять видів поганих файлів, недоступні посилання, помилка після відео (старий кадр не лишається), відновлення після помилки, обрізаний файл, порожній рядок
Список відтворення	6	додавання і навігація, межі списку, панель для одного і кількох файлів, автоперехід, повтор
Інше	8	діалоги (вибір і скасування), перетягування кількох файлів, клавіші, швидкість, повний екран, збереження кадру з відео і відмова для аудіо

Результат: `66 passed`. Набір прогнано 20 разів поспіль без жодного збою.

Що знайшли тести і багаторазові прогони

- У `QSlider` за замовчуванням діапазон від 0 до 99, тому без відкритого файлу при наведенні з'являлася підказка «00:00». Виправлено: смуга стартує з нульовим діапазоном.
- Підказка з таймкодом зникала за пів секунди: через цей час Qt надсилає смузі власну подію підказки, а в смуги вона порожня. Тепер смуга обробляє цю подію сама.
- Правило стилю для панелі перекривало стан наведення й увімкнення кнопок, і повтор чи список виглядали вимкненими. Помилку видно було лише на скриншоті; виправлено селектори.
- Перемикання екрана на заставку в `on_error` виявилося зайвим: при непридатному джерелі Qt сам надсилає `hasVideoChanged(False)`, і екран перемикається через наявний обробник. Рядок видалено, сценарій «грало відео, потім відкрито пошкоджений файл» перевіряє окремий тест.
- Спроба відкласти перемикання на наступний файл до наступного проходу циклу подій зробила гірше: Qt FFmpeg зависав у 8 випадках з 8, а синхронне перемикання в обробнику спрацювало 8 з 8. Залишено синхронний варіант.
- HTTP-сервер у потоці Python ділив GIL з Qt, і FFmpeg міг взаємно заблокуватися з ним. Сервер винесено в окремий процес.

Обмеження середовища. Розробка й тести йшли в Linux-контейнері без звукової карти. У ньому сам Qt Multimedia зависає в кінці MP4-файлу зі звуком AAC (це перевірено на «голому» `QMediaPlayer` без коду плеєра), тому тести автопереходу й повтору використовують WAV. На комп'ютері зі звичайним аудіовиходом цього обмеження бути не повинно.

9. Висновки

Розроблено desktop-медіаплеєр на Python і PySide6, який виконує всі вимоги варіанта 1: відкриває локальні файли і посилання, має Play/Pause/Stop, перемотування кнопками, клавішами і смугою прогресу, показ часу, гучність, підтримує 2 аудіо- і 3 відеоформати. Додатково реалізовано таймкод під курсором і перехід кліком по смузі, список відтворення з автопереходом, швидкість, повтор, повний екран, збереження кадру і зрозумілі повідомлення про помилки.

Програма перевірена на коректних, некоректних і граничних вхідних даних: порожні, пошкоджені й нетипові файли та недоступні посилання не призводять до падіння, а користувач бачить зрозумілу причину. Код складається з чистих функцій перетворення, класу смуги прогресу і класу вікна, де дії користувача йдуть у `QMediaPlayer`, а його сигнали повертаються в інтерфейс. Якість роботи підтверджують 66 автотестів на справжніх медіафайлах.